

---

## REQUIREMENTS NOT MET

---

None

---

## PROBLEMS ENCOUNTERED

---

None Encountered

---

## FUTURE WORK/APPLICATIONS

---

This lab teaches us the usage of Serial Peripheral Interface (SPI), which is fundamental to high-speed transmissions, by connecting a ‘master’ to a ‘student’ devices, we can have full-duplex communication from one to another, allowing us to have much faster rates than USART by sharing a clock signal (SCK). SPI also allows us to interface multiple devices or ‘students,’ by use of CS/SS. This could be used for high-speed sensors such as sound sensors, brightness sensors, etc. SPI is the ‘king of speed.’

---

## PRE-LAB EXERCISES

---

- i. In regard to SPI communication that is to exist between the relevant ATxmega128A1U and IMU chips, answer each of the questions within the previously given bulleted list.
  - a. Which device(s) should be given the role of master and which device(s) should be given the role of student?
    - i. The microcontroller should be given the master role to control the clock signal (SCK), chip selects, and the initialization of all read/write transmissions.
    - ii. The IMU chip should be given the student role to be driven by the microcontroller, remaining inactive except for when the master enables it via chip select and drives it with SCK, allowing it to transmit and be read from.
  - b. How will the student device(s) be enabled? If a student select is utilized, rather than just have the device(s) be permanently enabled, which pin(s) will be used?
    - i. The IMU will be enabled via an active-low CS/SS signal, which is driven by pin 12 for both chips.
  - c. What is the order of data transmission? Is the MSb or LSb transmitted first?
    - i. MSB in/out first down to LSB in/out last per IMU specification.
  - d. In regard to the relevant clock signal, should data be latched on a rising edge or on a falling edge?
    - i. It should be latched on the rising edge, per IMU timing diagram (page 25), with new data bits transmitted on the falling edge.
  - e. What is the maximum serial clock frequency that can be utilized by the relevant devices?
    - i. The IMU Timing Values chart shows a maximum SPI clock frequency of 10MHz.

---

## PSEUDOCODE/FLOWCHARTS

---

### spi.c

```
/*-----  
spi.c --  
  
Description:  
    Provides useful definitions for manipulating the relevant SPI  
    module of the ATxmega128A1U.  
  
Author(s): Dr. Eric M. Schwartz, Christopher Crary, Wesley Piard  
Last modified by: Dr. Eric M. Schwartz  
Last modified on: 30 May 2026  
-----*/  
  
/*****DEPENDENCIES*****/  
  
#include <avr/io.h>  
#include "spi.h"  
  
/*****END OF DEPENDENCIES*****/  
  
/*****FUNCTION DEFINITIONS*****/  
  
void spi_init(void)  
{  
  
    /* Initialize the relevant SPI output signals to be in an "idle" state.  
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.  
     * (You may wish to utilize the macros defined in `spi.h`.) */  
    PORTF.OUTSET = SS | SCK;  
  
    /* Configure the pin direction of relevant SPI signals. */  
    PORTF.DIRSET = MOSI | SS | SCK;  
    PORTF.DIRCLR = MISO;  
  
    /* Set the other relevant SPI configurations. */  
    SPIF.CTRL = SPI_PRESCALER_DIVX_gc |  
                SPI_MASTER_bm |  
                SPI_MODE_3_gc |  
                SPI_ENABLE_bm;  
}  
  
void spi_write(uint8_t data)  
{  
    /* Write to the relevant DATA register. */  
    SPIF.DATA = data;  
  
    /* Wait for relevant transfer to complete. */  
    while(IF FLAG NOT TRUE);  
  
    /* In general, it is probably wise to ensure that the relevant flag is  
     * cleared at this point, but, for our contexts, this will occur the  
     * next time we call the `spi_write` (or `spi_read`) routine.  
     * Really, because of how the flag must be cleared within  
     * ATxmega128A1U, it would probably make more sense to have some single  
     * function, say `spi_transceive`, that both writes and reads
```

```
    * data, rather than have two functions `spi_write` and `spi_read`,  
    * but we will not concern ourselves with this possibility  
    * during this semester of the course. */  
}  
  
uint8_t spi_read(void)  
{  
    /* Write some arbitrary data to initiate a transfer. */  
    SPIF.DATA = 0x37;  
  
    /* Wait for relevant transfer to be complete. */  
    while(IF FLAG NOT TRUE);  
  
    /* After the transmission, return the data that was received. */  
    return SPIF.DATA;  
}  
  
/*****END OF FUNCTION DEFINITIONS*****/
```

## Lab6\_2

```
/*
 * lab6_2.c
 *
 * Created: 7/12/2026 2:52:31 PM
 * Author : Evan Baesler
 */

#include <avr/io.h>
#include "spi.h"

int main(void)
{
    // Initialize the SPI module
    call spi_init

    while(1)
    {
        PORTF.OUTCLR = SS; // Simulate turning IMU CS on

        // Transmit 0x42
        spi_write(0x42)

        PORTF.OUTSET = SS; // Simulate turning IMU CS off
    }
}

/*****FUNCTION DEFINITIONS*****/

void spi_init(void)
{
    /* Initialize the relevant SPI output signals to be in an "idle" state.
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.
     * (You may wish to utilize the macros defined in `spi.h`.) */
    PORTF.OUTSET = SS | SCK;

    /* Configure the pin direction of relevant SPI signals. */
    PORTF.DIRSET = MOSI | SS | SCK;
    PORTF.DIRCLR = MISO;

    /* Set the other relevant SPI configurations. */
    SPIF.CTRL = SPI_PRESCALER_DIVX_gc |
                SPI_MASTER_bm |
                SPI_MODE_3_gc |
                SPI_ENABLE_bm;
}

void spi_write(uint8_t data)
{
    /* Write to the relevant DATA register. */
    SPIF.DATA = data;

    /* Wait for relevant transfer to complete. */
    while(IF FLAG NOT TRUE);

    /* In general, it is probably wise to ensure that the relevant flag is
     * cleared at this point, but, for our contexts, this will occur the
     * next time we call the `spi_write` (or `spi_read`) routine.
     */
}
```

```
    * Really, because of how the flag must be cleared within
    * ATxmega128A1U, it would probably make more sense to have some single
    * function, say `spi_transceive`, that both writes and reads
    * data, rather than have two functions `spi_write` and `spi_read`,
    * but we will not concern ourselves with this possibility
    * during this semester of the course. */
}

uint8_t spi_read(void)
{
    /* Write some arbitrary data to initiate a transfer. */
    SPIF.DATA = 0x37;

    /* Wait for relevant transfer to be complete. */
    while(IF FLAG NOT TRUE);

    /* After the transmission, return the data that was received. */
    return SPIF.DATA;
}

/*****END OF FUNCTION DEFINITIONS*****/
```

---

## PROGRAM CODE

---

```
spi.h
#ifndef SPI_H_                // Header guard.
#define SPI_H_

/*-----
spi.h --

Description:
    Provides function prototypes and macro definitions for utilizing the SPI
    system of the ATxmega128A1U.

Author(s): Dr. Eric M. Schwartz, Christopher Crary, Wesley Piard
Last modified by: Dr. Eric M. Schwartz
Last modified on: 30 May 2026
Last configured by: Evan Baesler
Last configured on: 12 July 2026
-----*/

/*****DEPENDENCIES*****/

#include <avr/io.h>

/*****END OF DEPENDENCIES*****/

/*****MACROS*****/

#define SS_bm    (1<<4)
#define MOSI_bm  (1<<5)
#define MISO_bm  (1<<6)
#define SCK_bm   (1<<7) // Per PORTF Alternate Functions

/*****END OF MACROS*****/

/*****FUNCTION PROTOTYPES*****/

/*-----
spi_init --

Description:
    Initializes the relevant SPI module to communicate with the LSM6DSL.

Input(s): N/A
Output(s): N/A
-----*/
void spi_init(void);

/*-----
spi_write --

Description:
    Transmits a single byte of data via the relevant SPI module.

Input(s): `data` - 8-bit value to be written via the relevant SPI module.
Output(s): N/A
-----*/
void spi_write(uint8_t data);

/*-----
```

`spi_read --`

Description:

Reads a byte of data via the relevant SPI module.

Input(s): N/A

Output(s): 8-bit value read from the relevant SPI module.

```
-----*/  
uint8_t spi_read(void);  
  
/*****END OF FUNCTION PROTOTYPES*****/  
  
#endif // End of header guard.
```

## spi.c

```
/*-----  
spi.c --  
  
Description:  
    Provides useful definitions for manipulating the relevant SPI  
    module of the ATxmega128A1U.  
  
Author(s): Dr. Eric M. Schwartz, Christopher Crary, Wesley Piard  
Last modified by: Dr. Eric M. Schwartz  
Last modified on: 30 May 2026  
Last configured by: Evan Baesler  
Last configured on: 12 July 2026  
-----*/  
  
/*****DEPENDENCIES*****/  
  
#include <avr/io.h>  
#include "spi.h"  
  
/*****END OF DEPENDENCIES*****/  
  
/*****FUNCTION DEFINITIONS*****/  
  
void spi_init(void)  
{  
  
    /* Initialize the relevant SPI output signals to be in an "idle" state.  
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.  
     * (You may wish to utilize the macros defined in `spi.h`.) */  
    PORTF.OUTSET = SS_bm | SCK_bm;  
  
    /* Configure the pin direction of relevant SPI signals. */  
    PORTF.DIRSET = SS_bm | MOSI_bm | SCK_bm;  
    PORTF.DIRCLR = MISO_bm;  
  
    /* Set the other relevant SPI configurations. */  
    SPIF.CTRL = SPI_PRESCALER_DIV16_gc |  
                SPI_MASTER_bm |  
                SPI_MODE_3_gc |  
                SPI_ENABLE_bm;  
    // This leaves CTRL.DORD false, so MSB -> LSB  
  
}  
  
void spi_write(uint8_t data)  
{  
    /* Write to the relevant DATA register. */  
    SPIF.DATA = data;  
  
    /* Wait for relevant transfer to complete. */  
    while(!(SPIF.STATUS & (SPI_IF_bm)));  
  
    /* In general, it is probably wise to ensure that the relevant flag is  
     * cleared at this point, but, for our contexts, this will occur the  
     * next time we call the `spi_write` (or `spi_read`) routine.  
     * Really, because of how the flag must be cleared within  
     * ATxmega128A1U, it would probably make more sense to have some single  
     * function, say `spi_transceive`, that both writes and reads  
     * data, rather than have two functions `spi_write` and `spi_read`,
```

```
    * but we will not concern ourselves with this possibility
    * during this semester of the course. */
}

uint8_t spi_read(void)
{
    /* Write some arbitrary data to initiate a transfer. */
    SPIF.DATA = 0x37;

    /* Wait for relevant transfer to be complete. */
    while(!(SPIF.STATUS & (SPI_IF_bm)));

    /* After the transmission, return the data that was received. */
    return SPIF.DATA;
}

/*****END OF FUNCTION DEFINITIONS*****/
```

## Lab6\_2

```
/*-----  
spi.c --  
  
Description:  
    Provides useful definitions for manipulating the relevant SPI  
    module of the ATxmega128A1U.  
  
Author(s): Dr. Eric M. Schwartz, Christopher Crary, Wesley Piard  
Last modified by: Dr. Eric M. Schwartz  
Last modified on: 30 May 2026  
Last configured by: Evan Baesler  
Last configured on: 12 July 2026  
-----*/  
  
/*****DEPENDENCIES*****/  
  
#include <avr/io.h>  
#include "spi.h"  
  
/*****END OF DEPENDENCIES*****/  
  
/*****FUNCTION DEFINITIONS*****/  
  
void spi_init(void)  
{  
  
    /* Initialize the relevant SPI output signals to be in an "idle" state.  
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.  
     * (You may wish to utilize the macros defined in `spi.h`.) */  
    PORTF.OUTSET = SS_bm | SCK_bm;  
  
    /* Configure the pin direction of relevant SPI signals. */  
    PORTF.DIRSET = SS_bm | MOSI_bm | SCK_bm;  
    PORTF.DIRCLR = MISO_bm;  
  
    /* Set the other relevant SPI configurations. */  
    SPIF.CTRL = SPI_PRESCALER_DIV16_gc |  
                SPI_MASTER_bm |  
                SPI_MODE_3_gc |  
                SPI_ENABLE_bm;  
    // This leaves CTRL.DORD false, so MSB -> LSB  
  
}  
  
void spi_write(uint8_t data)  
{  
    /* Write to the relevant DATA register. */  
    SPIF.DATA = data;  
  
    /* Wait for relevant transfer to complete. */  
    while(!(SPIF.STATUS & (SPI_IF_bm)));  
  
    /* In general, it is probably wise to ensure that the relevant flag is  
     * cleared at this point, but, for our contexts, this will occur the  
     * next time we call the `spi_write` (or `spi_read`) routine.  
     * Really, because of how the flag must be cleared within  
     * ATxmega128A1U, it would probably make more sense to have some single  
     * function, say `spi_transceive`, that both writes and reads
```

```
    * data, rather than have two functions `spi_write` and `spi_read`,  
    * but we will not concern ourselves with this possibility  
    * during this semester of the course. */  
}  
  
uint8_t spi_read(void)  
{  
    /* Write some arbitrary data to initiate a transfer. */  
    SPIF.DATA = 0x37;  
  
    /* Wait for relevant transfer to be complete. */  
    while(!(SPIF.STATUS & (SPI_IF_bm)));  
  
    /* After the transmission, return the data that was received. */  
    return SPIF.DATA;  
}  
  
/*****END OF FUNCTION DEFINITIONS*****/
```

APPENDIX



Figure 1: SPI Waveforms Capture

| Word Address  |   |              |                                               |
|---------------|---|--------------|-----------------------------------------------|
| ATxmega128A1U |   | ATxmega64A1U |                                               |
| 0             |   | 0            | Application Section (bytes)<br>(128K/64K)     |
|               |   |              |                                               |
|               |   |              | ...                                           |
| FFFF          | / | 77FF         | Application Table Section (bytes) (8K/<br>4K) |
| F000          | / | 7800         |                                               |
| FFFF          | / | 7FFF         |                                               |
| 10000         | / | 8000         | Boot Section (bytes)<br>(8K/4K)               |
| 10FFF         | / | 87FF         |                                               |

Figure 2: Program Memory Addressing

| Byte Address | ATxmega64A1U                | Byte Address | ATxmega128A1U               |
|--------------|-----------------------------|--------------|-----------------------------|
| 0            | I/O Registers (4K)          | 0            | I/O Registers (4KB)         |
| FFF          |                             | FFF          |                             |
| 1000         | EEPROM (2K)                 | 1000         | EEPROM (2K)                 |
| 17FF         |                             | 17FF         |                             |
|              | RESERVED                    |              | RESERVED                    |
| 2000         | Internal SRAM (4K)          | 2000         | Internal SRAM (8K)          |
| 2FFF         |                             | 3FFF         |                             |
| 4000         | External Memory (0 to 16MB) | 3000         | External Memory (0 to 16MB) |
| FFFFFF       |                             | FFFFFF       |                             |

Figure 3: Data Memory Addressing

### 13.16 Register summary – Ports

| Address | Name     | Bit 7        | Bit 6 | Bit 5    | Bit 4  | Bit 3        | Bit 2    | Bit 1        | Bit 0  | Page |
|---------|----------|--------------|-------|----------|--------|--------------|----------|--------------|--------|------|
| +0x00   | DIR      | DIR[7:0]     |       |          |        |              |          |              |        | 148  |
| +0x01   | DIRSET   | DIRSET[7:0]  |       |          |        |              |          |              |        | 148  |
| +0x02   | DIRCLR   | DIRCLR[7:0]  |       |          |        |              |          |              |        | 148  |
| +0x03   | DIRTGL   | DIRTGL[7:0]  |       |          |        |              |          |              |        | 148  |
| +0x04   | OUT      | OUT[7:0]     |       |          |        |              |          |              |        | 149  |
| +0x05   | OUTSET   | OUTSET[7:0]  |       |          |        |              |          |              |        | 149  |
| +0x06   | OUTCLR   | OUTCLR[7:0]  |       |          |        |              |          |              |        | 149  |
| +0x07   | OUTTGL   | OUTTGL[7:0]  |       |          |        |              |          |              |        | 149  |
| +0x08   | IN       | IN[7:0]      |       |          |        |              |          |              |        | 150  |
| +0x09   | INTCTRL  | –            | –     | –        | –      | INT1LVL[1:0] |          | INT0LVL[1:0] |        | 150  |
| +0x0A   | INT0MASK | INT0MSK[7:0] |       |          |        |              |          |              |        | 150  |
| +0x0B   | INT1MASK | INT1MSK[7:0] |       |          |        |              |          |              |        | 150  |
| +0x0C   | INTFLAGS | –            | –     | –        | –      | –            | –        | INT1IF       | INT0IF | 151  |
| +0x0D   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |
| +0x0E   | REMAP    | –            | –     | SPI      | USART0 | TC0D         | TC0C     | TC0B         | TC0A   | 151  |
| +0x0F   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |
| +0x10   | PIN0CTRL | SRLEN        | INVEN | OPC[2:0] |        |              | ISC[2:0] |              |        | 152  |
| +0x11   | PIN1CTRL | SRLEN        | INVEN | OPC[2:0] |        |              | ISC[2:0] |              |        | 152  |
| +0x12   | PIN2CTRL | SRLEN        | INVEN | OPC[2:0] |        |              | ISC[2:0] |              |        | 152  |
| +0x13   | PIN3CTRL | SRLEN        | INVEN | OPC[2:0] |        |              | ISC[2:0] |              |        | 152  |
| +0x14   | PIN4CTRL | SRLEN        | INVEN | OPC[2:0] |        |              | ISC[2:0] |              |        | 152  |
| +0x15   | PIN5CTRL | SRLEN        | INVEN | OPC[2:0] |        |              | ISC[2:0] |              |        | 152  |
| +0x16   | PIN6CTRL | SRLEN        | INVEN | OPC[2:0] |        |              | ISC[2:0] |              |        | 152  |
| +0x17   | PIN7CTRL | SRLEN        | INVEN | OPC[2:0] |        |              | ISC[2:0] |              |        | 152  |
| +0x18   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |
| +0x19   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |
| +0x1A   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |
| +0x1B   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |
| +0x1C   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |
| +0x1D   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |
| +0x1E   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |
| +0x1F   | Reserved | –            | –     | –        | –      | –            | –        | –            | –      |      |

Figure 4: Ports Register Summary

### 14.13 Register summary

| Address        | Name     | Bit 7         | Bit 6 | Bit 5         | Bit 4    | Bit 3         | Bit 2     | Bit 1          | Bit 0 | Page |
|----------------|----------|---------------|-------|---------------|----------|---------------|-----------|----------------|-------|------|
| +0x00          | CTRLA    | –             | –     | –             | –        | CLKSEL[3:0]   |           |                |       | 175  |
| +0x01          | CTRLB    | CCDEN         | CCCEN | CCBEN         | CCAEN    | –             | WGMD[2:0] |                |       | 175  |
| +0x02          | CTRLC    | –             | –     | –             | –        | CMPD          | CMPC      | CMPB           | CMPA  | 176  |
| +0x03          | CTRLD    | EVACT[2:0]    |       |               | EVDLY    | EVSEL[3:0]    |           |                |       | 176  |
| +0x04          | CTRLF    | –             | –     | –             | –        | –             | –         | BYTEM          |       | 178  |
| +0x05          | Reserved | –             | –     | –             | –        | –             | –         | –              | –     |      |
| +0x06          | INTCTRLA | –             | –     | –             | –        | ERRINTLV[1:0] |           | OVINTLV[1:0]   |       | 178  |
| +0x07          | INTCTRLB | CCCINTLV[1:0] |       | CCBINTLV[1:0] |          | CCAINTLV[1:0] |           | CCAVINTLV[1:0] |       | 178  |
| +0x08          | CTRLCLR  | –             | –     | –             | QDECINDX | CMD[1:0]      |           | LUPD           | DIR   | 179  |
| +0x09          | CTRLFSET | –             | –     | –             | QDECINDX | CMD[1:0]      |           | LUPD           | DIR   | 180  |
| +0x0A          | CTRLGCLR | –             | –     | –             | CCDBV    | CCCBV         | CCBBV     | CCABV          | PERBV | 180  |
| +0x0B          | CTRLGSET | –             | –     | –             | CCDBV    | CCCBV         | CCBBV     | CCABV          | PERBV | 180  |
| +0x0C          | INTFLAGS | CCDIF         | CCCIF | CCBIF         | CCAIF    | –             | –         | ERRIF          | OVFIF | 180  |
| +0x0D          | Reserved | –             | –     | –             | –        | –             | –         | –              | –     |      |
| +0x0E          | Reserved | –             | –     | –             | –        | –             | –         | –              | –     |      |
| +0x0F          | TEMP     | TEMP[7:0]     |       |               |          |               |           |                |       | 181  |
| +0x10 to +0x1F | Reserved | –             | –     | –             | –        | –             | –         | –              | –     |      |
| +0x20          | CNTL     | CNT[7:0]      |       |               |          |               |           |                |       | 181  |
| +0x21          | CNTH     | CNT[15:8]     |       |               |          |               |           |                |       | 181  |
| +0x22 to +0x25 | Reserved | –             | –     | –             | –        | –             | –         | –              | –     |      |
| +0x26          | PERL     | PER[7:0]      |       |               |          |               |           |                |       | 182  |
| +0x27          | PERH     | PER[8:15]     |       |               |          |               |           |                |       | 182  |
| +0x28          | CCAL     | CCA[7:0]      |       |               |          |               |           |                |       | 182  |
| +0x29          | CAAH     | CCA[15:8]     |       |               |          |               |           |                |       | 182  |
| +0x2A          | CCBL     | CCB[7:0]      |       |               |          |               |           |                |       | 182  |
| +0x2B          | CCBH     | CCB[15:8]     |       |               |          |               |           |                |       | 182  |
| +0x2C          | CCCL     | CCC[7:0]      |       |               |          |               |           |                |       | 182  |
| +0x2D          | CCCH     | CCC[15:8]     |       |               |          |               |           |                |       | 182  |
| +0x2E          | CCDL     | CCD[7:0]      |       |               |          |               |           |                |       | 182  |
| +0x2F          | CCDH     | CCD[15:8]     |       |               |          |               |           |                |       | 182  |
| +0x30 to +0x35 | Reserved | –             | –     | –             | –        | –             | –         | –              | –     |      |
| +0x36          | PERBUFL  | PERBUF[7:0]   |       |               |          |               |           |                |       | 183  |

Figure 5: Timer/Counter Register Summary

### 12.9 Register summary

| Address | Name   | Bit 7       | Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2   | Bit 1    | Bit 0   | Page |
|---------|--------|-------------|-------|-------|-------|-------|---------|----------|---------|------|
| +0x00   | STATUS | NMIEX       | –     | –     | –     | –     | HILVLEX | MEDLVLEX | LOLVLEX | 136  |
| +0x01   | INTPRI | INTPRI[7:0] |       |       |       |       |         |          |         | 136  |
| +0x02   | CTRL   | RREN        | IVSEL | –     | –     | –     | HILVLEN | MEDLVLEN | LOLVLEN | 137  |

Figure 6: Interrupts Register Summary

## 27.12 Register summary – EBI

| Address | Name       | Bit 7                              | Bit 6 | Bit 5                               | Bit 4 | Bit 3       | Bit 2          | Bit 1                     | Bit 0 | Page |
|---------|------------|------------------------------------|-------|-------------------------------------|-------|-------------|----------------|---------------------------|-------|------|
| +0x00   | CTRL       | SDDATAW[1:0]                       |       | LPCMODE[1:0]                        |       | SRMODE[1:0] |                | IFMODE[1:0]               |       | 329  |
| +0x01   | SDRAMCTRLA | –                                  | –     | –                                   | –     | SDCAS       | SDROW          | SDCOL[1:0]                |       | 330  |
| +0x02   | Reserved   | –                                  | –     | –                                   | –     | –           | –              | –                         | –     |      |
| +0x03   | Reserved   | –                                  | –     | –                                   | –     | –           | –              | –                         | –     |      |
| +0x04   | REFRESHL   | SDRAM Refresh Period Low Byte      |       |                                     |       |             |                |                           |       | 331  |
| +0x05   | REFRESHH   | –                                  | –     | –                                   | –     | –           | –              | SDRAM Refresh Period High |       | 331  |
| +0x06   | INITDLYL   | SDRAM Initialization Time Low Byte |       |                                     |       |             |                |                           |       | 331  |
| +0x07   | INITDLYH   | –                                  | –     | SDRAM Initialization Time High Byte |       |             |                |                           |       | 331  |
| +0x08   | SDRAMCTRLB | MRDLY[1:0]                         |       | ROWCYCDLY[[2:0]                     |       |             | RPDLY[2:0]     |                           |       | 332  |
| +0x09   | SDRAMCTRLC | WRDLY[1:0]                         |       | ESRDLY[2:0]                         |       |             | ROWCOLDLY[2:0] |                           |       | 333  |
| +0x0A   | Reserved   | –                                  | –     | –                                   | –     | –           | –              | –                         | –     |      |
| +0x0B   | Reserved   | –                                  | –     | –                                   | –     | –           | –              | –                         | –     |      |
| +0x0C   | Reserved   | –                                  | –     | –                                   | –     | –           | –              | –                         | –     |      |
| +0x0D   | Reserved   | –                                  | –     | –                                   | –     | –           | –              | –                         | –     |      |
| +0x0E   | Reserved   | –                                  | –     | –                                   | –     | –           | –              | –                         | –     |      |
| +0x0F   | Reserved   | –                                  | –     | –                                   | –     | –           | –              | –                         | –     |      |
| +0x10   | CS0        | Chip Select 0 Offset Address       |       |                                     |       |             |                |                           |       |      |
| +0x14   | CS1        | Chip Select 1 Offset Address       |       |                                     |       |             |                |                           |       |      |
| +0x18   | CS2        | Chip Select 2 Offset Address       |       |                                     |       |             |                |                           |       |      |
| +0x1C   | CS3        | Chip Select 3 Offset Address       |       |                                     |       |             |                |                           |       |      |

Figure 7: EBI Register Summary

## 27.13 Register summary – EBI chip select

| Address | Name      |         | Bit 7                              | Bit 6      | Bit 5 | Bit 4 | Bit 3 | Bit 2     | Bit 1       | Bit 0 | Page |     |
|---------|-----------|---------|------------------------------------|------------|-------|-------|-------|-----------|-------------|-------|------|-----|
| +0x00   | CTRLA     |         | –                                  | ASIZE[4:0] |       |       |       |           | MODE[1:0]   |       | 329  |     |
| +0x01   | CTRLB     | (SRAM)  | –                                  | –          | –     | –     | –     | SRWS[2:0] |             |       | 330  |     |
|         |           | (SDRAM) | SDINITDONE                         | –          | –     | –     | –     | SDSREN    | SDMODE[1:0] |       |      |     |
| +0x02   | BASEADDRL |         | Chip Select Base Address Low Byte  |            |       |       |       | –         | –           | –     | –    | 337 |
| +0x03   | BASEADDRH |         | Chip Select Base Address High Byte |            |       |       |       |           |             |       |      | 337 |

Figure 8: EBI Chip Select Register Summary

**Table 33-7.** Port H - alternate functions.

| PORT H | PIN # | INTERRUPT | SDRAM 3P         | SRAM ALE1            | SRAM ALE12        | LPC3 ALE1            | LPC2 ALE1         | LPC2 ALE12           |
|--------|-------|-----------|------------------|----------------------|-------------------|----------------------|-------------------|----------------------|
| GND    | 53    |           |                  |                      |                   |                      |                   |                      |
| VCC    | 54    |           |                  |                      |                   |                      |                   |                      |
| PH0    | 55    | SYNC      | $\overline{WE}$  | $\overline{WE}$      | $\overline{WE}$   | $\overline{WE}$      | $\overline{WE}$   | $\overline{WE}$      |
| PH1    | 56    | SYNC      | $\overline{CAS}$ | $\overline{RE}$      | $\overline{RE}$   | $\overline{RE}$      | $\overline{RE}$   | $\overline{RE}$      |
| PH2    | 57    | SYNC/ASYN | $\overline{RAS}$ | $\overline{ALE1}$    | $\overline{ALE1}$ | $\overline{ALE1}$    | $\overline{ALE1}$ | $\overline{ALE1}$    |
| PH3    | 58    | SYNC      | $\overline{DQM}$ |                      | $\overline{ALE2}$ |                      |                   | $\overline{ALE2}$    |
| PH4    | 59    | SYNC      | BA0              | $\overline{CS0}/A16$ | $\overline{CS0}$  | $\overline{CS0}/A16$ | $\overline{CS0}$  | $\overline{CS0}/A16$ |
| PH5    | 60    | SYNC      | BA1              | $\overline{CS1}/A17$ | $\overline{CS1}$  | $\overline{CS1}/A17$ | $\overline{CS1}$  | $\overline{CS1}/A17$ |
| PH6    | 61    | SYNC      | CKE              | $\overline{CS2}/A18$ | $\overline{CS2}$  | $\overline{CS2}/A18$ | $\overline{CS2}$  | $\overline{CS2}/A18$ |
| PH7    | 62    | SYNC      | CLK              | $\overline{CS3}/A19$ | $\overline{CS3}$  | $\overline{CS3}/A19$ | $\overline{CS3}$  | $\overline{CS3}/A19$ |

Notes: 1. CS0 - CS3 can optionally be moved to Port E or F  
2. A16-A23 can optionally be moved to Port E or F when EBI configured in 4PORT mode. Refer to the EBIOUT register description in the XMEGA AU Manual.

Figure 9: PORTH Alternate Functions

**Table 33-8.** Port J - alternate functions.

| PORT J | PIN # | INTERRUPT | SDRAM 3P | SRAM ALE1 | SRAM ALE12 | LPC3 ALE1 | LPC2 ALE1 | LPC2 ALE12 |
|--------|-------|-----------|----------|-----------|------------|-----------|-----------|------------|
| GND    | 63    |           |          |           |            |           |           |            |
| VCC    | 64    |           |          |           |            |           |           |            |
| PJ0    | 65    | SYNC      | D0       | D0        | D0         | D0/A0     | D0/A0     | D0/A0/A8   |
| PJ1    | 66    | SYNC      | D1       | D1        | D1         | D1/A1     | D1/A1     | D1/A1/A9   |

## ATxmega128A1U/ATxmega64A1U

| PORT J | PIN # | INTERRUPT | SDRAM 3P | SRAM ALE1 | SRAM ALE12 | LPC3 ALE1 | LPC2 ALE1 | LPC2 ALE12 |
|--------|-------|-----------|----------|-----------|------------|-----------|-----------|------------|
| PJ2    | 67    | SYNC/ASYN | D2       | D2        | D2         | D2/A2     | D2/A2     | D2/A2/A10  |
| PJ3    | 68    | SYNC      | D3       | D3        | D3         | D3/A3     | D3/A3     | D3/A3/A11  |
| PJ4    | 69    | SYNC      | A8       | D4        | D4         | D4/A4     | D4/A4     | D4/A4/A12  |
| PJ5    | 70    | SYNC      | A9       | D5        | D5         | D5/A5     | D5/A5     | D5/A5/A13  |
| PJ6    | 71    | SYNC      | A10      | D6        | D6         | D6/A6     | D6/A6     | D6/A6/A14  |
| PJ7    | 72    | SYNC      | A11      | D7        | D7         | D7/A7     | D7/A7     | D7/A7/A15  |

Figure 10: PORTJ Alternate Functions

**Table 33-9.** Port K - alternate functions.

| PORT K | PIN # | INTERRUPT  | SDRAM 3P | SRAM ALE1 | SRAM ALE2  | LPC3 ALE1 |
|--------|-------|------------|----------|-----------|------------|-----------|
| GND    | 73    |            |          |           |            |           |
| VCC    | 74    |            |          |           |            |           |
| PK0    | 75    | SYNC       | A0       | A0/A8     | A0/A8/A16  | A8        |
| PK1    | 76    | SYNC       | A1       | A1/A9     | A1/A9/A17  | A9        |
| PK2    | 77    | SYNC/ASYNC | A2       | A2/A10    | A2/A10/A18 | A10       |
| PK3    | 78    | SYNC       | A3       | A3/A11    | A3/A11/A19 | A11       |
| PK4    | 79    | SYNC       | A4       | A4/A12    | A4/A12/A20 | A12       |
| PK5    | 80    | SYNC       | A5       | A5/A13    | A5/A13/A21 | A13       |
| PK6    | 81    | SYNC       | A6       | A6/A14    | A6/A14/A22 | A14       |
| PK7    | 82    | SYNC       | A7       | A7/A15    | A7/A15/A23 | A15       |

Figure 11: PORTK Alternate Functions

## 23.16 Register summary

### 23.16.1 Register description – USART

| Address | Name      | Bit 7       | Bit 6 | Bit 5          | Bit 4 | Bit 3          | Bit 2       | Bit 1          | Bit 0 | Page |
|---------|-----------|-------------|-------|----------------|-------|----------------|-------------|----------------|-------|------|
| +0x00   | DATA      | DATA[7:0]   |       |                |       |                |             |                |       | 294  |
| +0x01   | STATUS    | RXCIF       | TXCIF | DREIF          | FERR  | BUOVF          | PERR        | –              | RXB8  | 294  |
| +0x02   | Reserved  | –           | –     | –              | –     | –              | –           | –              | –     |      |
| +0x03   | CTRLA     | –           | –     | RXCINTLVL[1:0] |       | TXCINTLVL[1:0] |             | DREINTLVL[1:0] |       | 295  |
| +0x04   | CTRLB     | –           | –     | –              | RXEN  | TXEN           | CLK2X       | MPCM           | TXB8  | 296  |
| +0x05   | CTRLC     | CMODE[1:0]  |       | PMODE[1:0]     |       | SBMODE         | CHSIZE[2:0] |                |       | 296  |
| +0x06   | BAUDCTRLA | BSEL[7:0]   |       |                |       |                |             |                |       | 298  |
| +0x07   | BAUDCTRLB | BSCALE[3:0] |       |                |       | BSEL[11:8]     |             |                |       | 298  |

### 23.16.2 Register description – USART in SPI Master Mode

| Address | Name      | Bit 7       | Bit 6 | Bit 5          | Bit 4 | Bit 3          | Bit 2 | Bit 1          | Bit 0 | Page |
|---------|-----------|-------------|-------|----------------|-------|----------------|-------|----------------|-------|------|
| +0x00   | DATA      | DATA[7:0]   |       |                |       |                |       |                |       | 294  |
| +0x01   | STATUS    | RXCIF       | TXCIF | DREIF          | –     | –              | –     | –              | –     | 294  |
| +0x02   | Reserved  | –           | –     | –              | –     | –              | –     | –              | –     |      |
| +0x03   | CTRLA     | –           | –     | RXCINTLVL[1:0] |       | TXCINTLVL[1:0] |       | DREINTLVL[1:0] |       | 295  |
| +0x04   | CTRLB     | –           | –     | –              | RXEN  | TXEN           | –     | –              | –     | 296  |
| +0x05   | CTRLC     | CMODE[1:0]  |       | –              | –     | –              | UDORD | UCPHA          | –     | 296  |
| +0x06   | BAUDCTRLA | BSEL[7:0]   |       |                |       |                |       |                |       | 298  |
| +0x07   | BAUDCTRLB | BSCALE[3:0] |       |                |       | BSEL[11:8]     |       |                |       | 298  |

Figure 12: USART Register Summaries

**Table 33-3.** Port C - alternate functions.

| PORT C | PIN# | INTERRUPT | TCC0 <sup>(1)(2)</sup> | AWEXC  | TCC1 | USARTC0 <sup>(3)</sup> | USARTC1 | SPIC <sup>(4)</sup> | TWIC | CLOCKOUT <sup>(5)</sup> | EVENTOUT <sup>(6)</sup> |
|--------|------|-----------|------------------------|--------|------|------------------------|---------|---------------------|------|-------------------------|-------------------------|
| GND    | 13   |           |                        |        |      |                        |         |                     |      |                         |                         |
| VCC    | 14   |           |                        |        |      |                        |         |                     |      |                         |                         |
| PC0    | 15   | SYNC      | OC0A                   | OC0ALS |      |                        |         |                     | SDA  |                         |                         |

## ATxmega128A1U/ATxmega64A1U

| PORT C | PIN# | INTERRUPT  | TCC0 <sup>(1)(2)</sup> | AWEXC  | TCC1 | USARTC0 <sup>(3)</sup> | USARTC1 | SPIC <sup>(4)</sup> | TWIC | CLOCKOUT <sup>(5)</sup> | EVENTOUT <sup>(6)</sup> |
|--------|------|------------|------------------------|--------|------|------------------------|---------|---------------------|------|-------------------------|-------------------------|
| PC1    | 16   | SYNC       | OC0B                   | OC0AHS |      | XCK0                   |         |                     | SCL  |                         |                         |
| PC2    | 17   | SYNC/ASYNC | OC0C                   | OC0BLS |      | RXD0                   |         |                     |      |                         |                         |
| PC3    | 18   | SYNC       | OC0D                   | OC0BHS |      | TXD0                   |         |                     |      |                         |                         |
| PC4    | 19   | SYNC       |                        | OC0CLS | OC1A |                        |         | SS                  |      |                         |                         |
| PC5    | 20   | SYNC       |                        | OC0CHS | OC1B |                        | XCK1    | MOSI                |      |                         |                         |
| PC6    | 21   | SYNC       |                        | OC0DLS |      |                        | RXD1    | MISO                |      | clk <sub>RTC</sub>      |                         |
| PC7    | 22   | SYNC       |                        | OC0DHS |      |                        | TXD1    | SCK                 |      | clk <sub>PER</sub>      | EVOUT                   |

- Notes:
1. Pin mapping of all TC0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
  2. If TC0 is configured as TC2 all eight pins can be used for PWM output. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
  3. Pin mapping of all USART0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
  4. Pins MOSI and SCK for all SPI can optionally be swapped. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
  5. CLKOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.
  6. EVOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.

Figure 13: PORTC Alternate Functions

Notes: 1. All pins on the port can optionally be used for EBI chip select or address lines. Refer to the EBIOUT register description in the XMEGA AU Manual.

**Table 33-6. Port F - alternate functions.**

| PORT F | PIN # | INTERRUPT  | TCF0 | TCF1 | USARTF0 | USARTF1 | SPIF            | TWIF |
|--------|-------|------------|------|------|---------|---------|-----------------|------|
| GND    | 43    |            |      |      |         |         |                 |      |
| VCC    | 44    |            |      |      |         |         |                 |      |
| PF0    | 45    | SYNC       | OC0A |      |         |         |                 | SDA  |
| PF1    | 46    | SYNC       | OC0B |      | XCK0    |         |                 | SCL  |
| PF2    | 47    | SYNC/ASYNC | OC0C |      | RXD0    |         |                 |      |
| PF3    | 48    | SYNC       | OC0D |      | TXD0    |         |                 |      |
| PF4    | 49    | SYNC       |      | OC1A |         |         | $\overline{SS}$ |      |
| PF5    | 50    | SYNC       |      | OC1B |         | XCK1    | MOSI            |      |
| PF6    | 51    | SYNC       |      |      |         | RXD1    | MISO            |      |
| PF7    | 52    | SYNC       |      |      |         | TXD1    | SCK             |      |

Note: 1. All pins on the port can optionally be used for EBI chip select or address lines. Refer to the EBIOUT register description in the XMEGA AU Manual.

Figure 14: PORTF Alternate Functions

## 22.8 Register summary

| Address | Name    | Bit 7     | Bit 6  | Bit 5 | Bit 4  | Bit 3     | Bit 2 | Bit 1          | Bit 0 | Page                |
|---------|---------|-----------|--------|-------|--------|-----------|-------|----------------|-------|---------------------|
| +0x00   | CTRL    | CLK2X     | ENABLE | DORD  | MASTER | MODE[1:0] |       | PRESCALER[1:0] |       | <a href="#">276</a> |
| +0x01   | INTCTRL | –         | –      | –     | –      | –         | –     | INTLVL[1:0]    |       | <a href="#">277</a> |
| +0x02   | STATUS  | IF        | WRCOL  | –     | –      | –         | –     | –              | –     | <a href="#">277</a> |
| +0x03   | DATA    | DATA[7:0] |        |       |        |           |       |                |       | <a href="#">278</a> |

## 22.9 Interrupt vector summary

**Table 22-4. SPI interrupt vector and its offset word address.**

| Offset | Source   | Interrupt Description |
|--------|----------|-----------------------|
| 0x00   | SPI_vect | SPI interrupt vector  |

Figure 15: SPI Register Summary